<table>
<tr><th>Question</th><th></th><th></th><th>Marks</th></tr>
<tr><td>11</td><td>1</td><td>

All marks for AO3 (programming)

1. Iterative structure contains code that gets the second position from the player;
2. One correct condition;
3. Both correct conditions and correct logic;
4. Displays error message under all correct circumstances and only under correct circumstances; **R.** message same as original prompt

Max 3 if code contains errors

NOTE: correct conditions are:
`EndRow >= StartRow` and `EndColumn >= StartColumn`
A. reversed conditions if appropriate.

</td><td>4</td></tr>
<tr><td>11</td><td>2</td><td>

Mark is for AO3 (evaluate)

***** SCREEN CAPTURE *****
*Must match code from **11.1**.*
*Code for **11.1** must be sensible.*

Screen capture showing message displayed when `1` then `4` is entered followed by `4` then `4` being entered and accepted.

```
Enter simulation number: 1

1. Display overall details
2. Display area details
3. Inspect cell
4. Advance one stage
5. Advance X stages
9. Quit

> 2

Enter row number: 2
Enter column number: 2


Enter row number: 1
Enter column number: 4

Invalid position. Please enter another.

Enter row number: 4
Enter column number: 4

2, 2:
2, 3:
2, 4: | Nest |  | Ants: 5 |
3, 2:
3, 3:
3, 4:
4, 2: | 500 food |
```

</td><td>1</td></tr>
</table>

11	**Typical Solution:**
	
	...
	switch (Choice)
	{
	  case "1":
	  Console.WriteLine(ThisSimulation.GetDetails());
	  break;
	  case "2":
	  int StartRow = 0, StartColumn = 0, EndRow = 0, EndColumn = 0;
	  GetCellReference(ref StartRow, ref StartColumn);
	  //New Code:
	  while (true)
	  {
	  GetCellReference(ref EndRow, ref EndColumn);
	  if (EndRow >= StartRow && EndColumn >= StartColumn) break;
	  Console.WriteLine("Invalid position. Please enter another.");
	  }
	  // End of new code
	  Console.WriteLine(ThisSimulation.GetAreaDetails(StartRow, StartColumn, EndRow, EndColumn));
	  break;
	  case "3":
	  ...	

Question			Marks
12	1	**All marks for AO3 (programming)** **Marks 1 and 2 refer to modifications to the main program** 1. New main menu option with sensible message; **A.** suitable alternative message 2. Main selection structure updated to call ThisSimulation.Diapause appropriately; **Marks 3 to 5 refer to modifications to the Simulation class** 3. Method added to `Simulation` with the correct interface; **I.** case and minor typos 4. Iterative structure repeats once for each `Ant` in `Ants`; 5. Call to `GoHome` method for each ant; **A.** Call is only made for WorkerAnts **Marks 6 and 7 refer to modifications to the Ant class** 6. `GoHome` method with right interface in `Ant` **R.** just for WorkerAnt **A.** in WorkerAnt and QueenAnt 7. Ant Row and Column set to NestRow and NestColumn appropriately.	7
12	2	**Mark is for AO3 (evaluate)** ****** SCREEN CAPTURE ****** *Must match code from **12.1**, including prompts on screen capture matching those in code.* *Code for **12.1** must be sensible.* Screen capture(s) showing ants away from the nest and **all** ants later in the nest. 	1

```
1. Display overall details
2. Display area details
3. Inspect cell
4. Advance one stage
5. Advance X stages
6. Diapause
9. Quit

> 6

1. Display overall details
2. Display area details
3. Inspect cell
4. Advance one stage
5. Advance X stages
6. Diapause
9. Quit

> 1
1, 1:
1, 2:
1, 3:
1, 4: | Pheromones: 1 |  | 474 food |
1, 5:
2, 1:
2, 2:
2, 3:
2, 4: | Nest |  | Ants: 6 |
2, 5:
3, 1:
3, 2: | 500 food |
3, 3:
3, 4: | Pheromones: 1 |  | 495 food |
3, 5:
4, 1:
4, 2:
4, 3:
4, 4:
4, 5:
5, 1:
5, 2:
5, 3:
5, 4:
5, 5:
```

Expected Solution:

Changes to `DisplayMenu`:

```
static void DisplayMenu()
{
    Console.WriteLine();
    Console.WriteLine("1. Display overall details");
    Console.WriteLine("2. Display area details");
    Console.WriteLine("3. Inspect cell");
    Console.WriteLine("4. Advance one stage");
    Console.WriteLine("5. Advance X stages");
    Console.WriteLine("6. Diapause");
    Console.WriteLine("9. Quit");
    Console.WriteLine();
    Console.Write("> ");
}
```

Changes to `Main`

```
...
case "6":
    ThisSimulation.Diapause();
    break;
...
```

Diapause Method in Simulation class

```
public void Diapause()
{
    foreach(Ant A in Ants)
    {
        A.GoHome();
    }
}
```

GoHome Method in Ant class

```
public void GoHome()
{
    Row = NestRow;
    Column = NestColumn;
}
```

Question			Marks
13	1	**All marks for AO3 (programming)** Mark points 1 and 2 relate to the `SeekerAnt` class. 1. Creating a new class called `SeekerAnt` that inherits from `WorkerAnt`; **R.** other names for class **I.** case and minor typos 2. Constructor that calls parent class constructor and then sets `TypeOfAnt` to `seeker` && `FootCapacity` to 5. 3. Method `SeekFood` created with correct interface. 4. Iterative structure that checks each cell within the grid. 5. Selection structures comparing each cell's food amount to 0. 6. Distance between Ant and a Cell is correctly calculated. 7. Cell with lowest Manhattan Distance is identified. 8. Ant moves towards this cell. Mark point 9 refers to the `AdvanceStage` method within the `Simulation` class. 9. Selection structure calls `SeekerAnts` `SeekFood` method in the correct location; **R.** If Seeker Ant is unable to move correctly under all circumstances. Mark points 10 and 11 relate to the `SetUpNestAt` method within the `Simulation` class. 10. Modified `SetUpANestAt` so that nests are set up with an additional seeker ant. 11. Modified `SetUpANestAt` so that one fewer worker ants are created.	11

13	**2**	**Mark is for AO3 (evaluate)** ****** SCREEN CAPTURE ****** *Must match code from **13.1**, including prompts on screen capture matching those in code.* *Code for **13.1** must be sensible.* Screen capture(s) showing at least one ant paths directly towards the nearest food. **Expected Solution**: Seeker Class	1

```
class SeekerAnt : WorkerAnt
{
    public SeekerAnt(int Row, int Column) : base(Row, Column, Row, Column) {
        TypeOfAnt = "seeker";
        FoodCapacity = 5;
    }
    public void SeekFood(List<Cell> grid)
    {
        Cell bestCell = null;
        int bestDistance = int.MaxValue;
        foreach (Cell c in grid)
        {
            if (c.GetAmountOfFood() == 0) continue;

            int distance = Math.Abs(c.GetColumn() - Column) +
Math.Abs(c.GetRow() - Row);
            if (distance < bestDistance)
            {
                bestDistance = distance;
                bestCell = c;
            }
        }
        if (Row < bestCell.GetRow())
        {
            Row++;
        } else if (Row > bestCell.GetRow())
        {
            Row--;
        }
        if (Column < bestCell.GetColumn())
        {
            Column++;
        }
        else if (Column > bestCell.GetColumn())
        {
            Column--;
        }
    }
}
```

Modifications to Simulation

```
public void SetUpANestAt(int Row, int Column)
{
    Nests.Add(new Nest(Row, Column, StartingFoodInNest));
    Ants.Add(new QueenAnt(Row, Column, Row, Column));
    Ants.Add(new SeekerAnt(Row, Column));
    for (int Worker = 3; Worker <= StartingAntsInNest; Worker++)
    {
        Ants.Add(new WorkerAnt(Row, Column, Row, Column));
    }
}
```

Modifications to AdvanceStage

```
public void AdvanceStage(int NumberOfStages)
{
    for (int Count = 1; Count <= NumberOfStages; Count++)
    {
      ...

        if (A.GetFoodCarried() > 0)
        {
            UpdateAntsPheromoneInCell(A);
        }
        if (A.GetFoodCarried() > 0 || A.GetTypeOfAnt() != "seeker"){
            A.ChooseCellToMoveTo(GetIndicesOfNeighbours(A.GetRow(),
A.GetColumn()),

GetIndexOfNeighbourWithStrongestPheromone(A.GetRow(), A.GetColumn()));
        } else
        {
            ((SeekerAnt)A).SeekFood(Grid);
        }
        ...
    }
}
```

Question			Marks
14	1	**All marks for AO3 (programming)** 1. Random location in grid generated within the main iterative structure; 2. Iterative structure checks each ant; **A.** skips Queen ants 3. Selection structure in loop checks location 4. Checks queen; 5. Selection to compare number of ants in cell with 0; 6. Random from 1 to 5; 7. Random number never greater than number of ants in cell. 8. Iterative structure with correct conditions; 9. Random ant is removed; 10. Correct number of ants are removed; **R.** Same ant can be removed multiple times. 11. Printout stating the number of ants eaten and the location.	11

14	**2**	**Mark is for AO3 (evaluate)**
		
		****** SCREEN CAPTURE ******
		*Must match code from **14.1**, including prompts on screen capture matching those in code.*
		*Code for **14.1** must be sensible.*
		
		Screen capture(s) showing the user running 100 stages, and the simulation terminating early.
		
		`Enter simulation number: 1`
		
		`1. Display overall details`
		`2. Display area details`
		`3. Inspect cell`
		`4. Advance one stage`
		`5. Advance X stages`
		`9. Quit`
		
		`> 5`
		`Enter number of stages to advance by: 10`
		`The bird ate 0 ants at position 1,3`
		`The bird ate 1 ants at position 2,5`
		`The bird ate 0 ants at position 1,2`
		`The bird ate 0 ants at position 4,3`
		`The bird ate 0 ants at position 5,1`
		`The bird ate 0 ants at position 2,5`
		`The bird ate 0 ants at position 1,2`
		`The bird ate 0 ants at position 5,3`
		`The bird ate 2 ants at position 2,4`
		`The bird ate 0 ants at position 2,2`
		`Simulation moved on 10 stages`
		
		`1. Display overall details`
		`2. Display area details`
		`3. Inspect cell`
		`4. Advance one stage`
		`5. Advance X stages`
			1

Expected Solution:
Changes to `Simulation.AdvanceStage`:

```
public void AdvanceStage(int NumberOfStages)
{
    for (int Count = 1; Count <= NumberOfStages; Count++)
    {

        int birdRow = RGen.Next(1, NumberOfRows + 1);
        int birdColumn = RGen.Next(1, NumberOfColumns + 1);

            List<Ant> AntsInCell = new List<Ant>();
            foreach (Ant A in Ants)
            {
                if (A.GetRow() == birdRow && A.GetColumn() == birdColumn &&
A.GetTypeOfAnt() !="queen")
                {
                    AntsInCell.Add(A);
                }
            }
        int antsToEat = 0;
        if (AntsInCell.Count() > 0)
        {
            antsToEat = rnd.Next(1, 6);
            if(antsToEat>AntsInCell.Count())
            {
                antsToEat = AntsInCell.Count();
            }

            for (int A = 1; A <= antsToEat; A++)
            {
                Ant AntToEat = AntsInCell[rnd.Next(0, AntsInCell.Count())];
                Ants.Remove(AntToEat);
                AntsInCell.Remove(AntToEat);
            }

        }
        Console.WriteLine($"The bird ate {antsToEat} ants at position {birdRow},
{birdColumn}");

        List<Pheromone> PheromonesToDelete = new List<Pheromone>();

        ...
```